

FDD3024 Hoare logic 2: variants and applications

Karl Palmskog

KTH

Tuesday May 19, 2026

Theory-oriented Hoare logic

- idealized programming language
- defined for language statements
- pre-allocation of all data
- no automated generation of verification conditions (VCs)
- no automation for discharging VCs

Practical Hoare logic

- defined at program level
- supports procedures/methods/functions
- supports recursion
- automated generation of VCs
- discharging of VCs by solvers (SAT, SMT, ...)
- intuitive interface to programmers

Compositionality

- compose components to make a whole program
- rule out “interference” between components
- program correctness uses components properties
- used in “verified component integration”
- used in “correct by construction”

Example: ACSL and C

```
/*@ requires min < max && min <= timer <= max;
    ensures
        (cond == 0 ==> \result == MAX(min, \old(timer) - 1)) &&
        (cond != 0 ==> \result == MIN(max, \old(timer) + 1)); */
static int update_sat(char cond, int timer, int min, int max) {
    int res = timer;
    if (cond) {res += 1;} else {res -= 1;}
    if (res > max) res = max;
    if (res < min) res = min;
    return res;
}
```

C code using function

```
static int timer;

void tmon_step(void){
    int sens = tmon_sens_read();
    timer = update_sat(sens, timer, 0, 10);
    if (timer == 10) {
        // activate warning
        tmon_warn_write(1);
    } else if (timer == 0) {
        // deactivate warning
        tmon_warn_write(0);
    } else { }
}
```

Modular verification with Hoare logic

- write top level specification of “main” procedure
- write specifications of auxiliary procedures used
- verify each procedure in isolation
 - assume other specifications are verified
 - no need to “look into” other code
- obtain main procedure correctness
- scales linearly with number of procedures

Modular verification vs inlining

- significant overhead from procedures
- significant overhead from big code
- tradeoff is hard to tell from the outset
- verifier optimized

Procedures and contracts

- Hoare logic when applied to procedures is a “contract”
- if callers meet preconditions, they get postcondition on return
- termination may or may not be guaranteed
- if precondition is not true, “anything” could happen

Contract design

- to facilitate verified programs, need easy-to-meet contracts
- procedure precondition should be easy to establish
- postcondition should be strong and usable

Predicates vs. Functional model

- contract often expressed in terms of variable values
- but may not capture intent of procedure
- ideal procedure may be on mathematical integers
- but program only has machine integers

Machine integers vs. math integers

- binary search in Java was broken for many years
- discrepancy between algorithm and implementation
- Search for “extra extra binary search”

Broken Java version

```
public static int binarySearch(int[] a, int key) {  
    int low = 0;  
    int high = a.length - 1;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        int midVal = a[mid];  
        if (midVal < key) {  
            low = mid + 1;  
        } else if (midVal > key) {  
            high = mid - 1;  
        } else {  
            return mid;  
        }  
    }  
    return -(low + 1);  
}
```

Fixed Java version

```
public static int binarySearch(int[] a, int key) {  
    int low = 0;  
    int high = a.length - 1;  
    while (low <= high) {  
        int mid = low + ((high - low) / 2);  
        int midVal = a[mid];  
        if (midVal < key) {  
            low = mid + 1;  
        } else if (midVal > key) {  
            high = mid - 1;  
        } else {  
            return mid;  
        }  
    }  
    return -low - 1;  
}
```

Specification as JML contract

```
/*@ requires
    @   a != null && sorted(a, 0, a.length - 1);
    @
    @ assigns
    @   \nothing;
    @
    @ behavior has_key:
    @   assumes
    @       \exists integer i; 0 <= i <= a.length - 1 && a[i] == key;
    @   ensures
    @       a[\result] == key;
    @
    @ behavior has_not_key:
    @   assumes
    @       \forall integer i; 0 <= i <= a.length - 1 ==> a[i] != key;
    @   ensures
    @       insertion_point(-\result - 1, key, a);
    @*/
public static int binarySearch(int[] a, int key)
```

Verified JML annotated fixed version

```
int low = 0; int high = a.length - 1;
/*@ loop_invariant 0 <= low && high <= a.length - 1;
    @ for has_key:
    @   loop_invariant
    @     \exists integer i; low <= i <= high && a[i] == key;
    @ for has_not_key:
    @   loop_invariant
    @     low <= a.length &&
    @       (\forall integer i; 0 <= i < low ==> a[i] < key) &&
    @       (\forall integer i; high < i <= a.length - 1 ==> key < a[i]);
    @ loop_variant high - low;
    @*/
while (low <= high) {
    int mid = low + ((high - low) / 2);
    int midVal = a[mid];
    if (midVal < key) {
        low = mid + 1;
    } else if (midVal > key) { high = mid - 1; }
    else { return mid; }
}
return -low - 1;
```


Functional model basics

- piece of memory is interpreted as mathematical structure
- procedure that mutate memory lead to “new” structure
- the mathematical meaning is always preserved
- often more concise than “variable properties” approach

Ghost variables

- ghost variables are program variables unused by computations
- often used to assist Hoare-style verification
- initialized and updated like other variables
- “real” program variables can’t depend on ghost ones
- bookkeeping for verification engineers

Heap memory

- if we only use stack memory, procedure could be pure function
- heap memory persists across procedure calls
- we need heap structure assumptions to get work done
- big research topic, many different approaches

Dafny and heap memory

- Dafny is a verification-oriented language
- translates to C#
- pre- and postconditions (contracts) built in
- usual programming constructs, invariants, etc.
- procedures state memory they change and how

Dafny ArrayList

```
class ArrayList<T> {  
  
    var arr: array<T>;  
    var size: int;  
  
    ghost var list: seq<T>;  
    ghost var Repr: set<object>;  
  
    // ...  
  
}
```

Valid ArrayList

```
predicate Valid()  
  reads this, Repr;  
{  
  Repr == {this, arr} &&  
  |list| == size &&  
  size <= arr.Length &&  
  forall i :: 0 <= i < size ==> arr[i] == list[i]  
}
```

ArrayList constructor

```
constructor InitialCapacity(initialCapacity: int,  
    archetype: T)  
    requires initialCapacity >= 0;  
    ensures Valid() && fresh(Repr);  
    ensures size == 0;  
    ensures list == [];  
    ensures arr.Length == initialCapacity;  
{  
    Repr := {this};  
    arr := new T[initialCapacity](_ => archetype);  
    size := 0;  
    list := [];  
    Repr := Repr + {arr};  
}
```

ArrayList getters

```
method Size() returns (lSize: int)
  requires Valid();
  ensures lSize == |list|;
{ lSize := size; }
```

```
method IsEmpty() returns (lEmpty: bool)
  requires Valid();
  ensures lEmpty <==> |list| == 0;
{ lEmpty := size == 0; }
```

```
method Get(index: int) returns (element: T)
  requires Valid();
  requires 0 <= index < size;
  ensures element == list[index];
{ element := arr[index]; }
```


ArrayList EnsureCapacity

```
method EnsureCapacity(minCapacity: int, archetype: T)
  modifies Repr; requires Valid(); ensures Valid();
  ensures list == old(list);
  ensures minCapacity > old(arr.Length) ==> fresh(Repr - {this});
  ensures minCapacity <= old(arr.Length) ==> Repr == old(Repr);
  ensures arr.Length >= minCapacity;
{
  var oldCapacity := arr.Length;
  if minCapacity > oldCapacity {
    var newCapacity := (oldCapacity * 3)/2 + 1;
    if (newCapacity < minCapacity) {
      newCapacity := minCapacity;
    }
    var size', arr' := size, arr;
    var newArr := new T[newCapacity](i reads arr'
      requires 0 <= i => if i < size'
      then arr'[i] else archetype);
    Repr := Repr - {arr} + {newArr};
    arr := newArr;
  }
}
```

ArrayList Add

```
method Add(element: T) returns (index: int)
  modifies Repr;
  requires Valid();
  ensures Valid();
  ensures size == old(size) + 1;
  ensures index == old(size);
  ensures list[index] == element;
  ensures list == old(list) + [element];
  ensures forall k :: 0 <= k < index ==> list[k] == old(list[k]);
  ensures old(arr.Length) < old(size) + 1 ==>
    fresh(Repr - {this}) && arr.Length >= old(size) + 1;
  ensures old(arr.Length) >= old(size) + 1 ==> Repr == old(Repr);
{
  EnsureCapacity(size + 1, element);
  arr[size] := element;
  list := list + [element];
  index := size;
  size := size + 1;
}
```

ArrayList Remove

```
method Remove(index: int)
  modifies Repr; requires Valid(); requires 0 <= index < size;
  ensures Valid(); ensures size == old(size) - 1;
  ensures list == old(list[0..index]) + old(list[index + 1..size]);
  ensures forall k :: 0 <= k < index ==> list[k] == old(list[k]);
  ensures forall k :: index <= k < size ==> list[k] == old(list[k + 1]);
  ensures Repr == old(Repr);
{
  ghost var l := list[0..index]; var i := index;
  while i < size - 1
    modifies arr;
    invariant index <= i < size; invariant |l| == i;
    invariant l == list[0..index] + list[index + 1..i + 1];
    invariant forall j :: index <= j < i ==> arr[j] == old(arr[j+1])
    invariant forall j :: 0 <= j < i ==> arr[j] == l[j];
    invariant forall j :: i < j < arr.Length ==> arr[j] == old(arr[j]);
  {
    arr[i] := arr[i+1]; l := l + [arr[i]]; i := i + 1;
  }
  list := l; size := size - 1;
}
```

ArrayList Append

```
method Append(a: ArrayList<T>)
  modifies Repr; requires Valid(); requires a.Valid();
  requires Repr !! a.Repr;
  ensures Valid();
  ensures size == old(size) + a.size;
  ensures list == old(list) + a.list;
  ensures forall k :: 0 <= k < old(size) ==>
    list[k] == old(list[k]);
  ensures forall k :: 0 <= k < a.size ==>
    list[old(size) + k] == a.list[k];
  ensures old(arr.Length) < old(size) + a.size ==> fresh(Repr - {this});
  ensures old(arr.Length) >= old(size) + a.size ==> Repr == old(Repr);
{
  var numNew := a.size;
  if numNew != 0 {
    EnsureCapacity(size + numNew, a.arr[0]);
    ArrayCopy(a.arr, 0, arr, size, numNew);
    list := list + a.list;
    size := size + numNew;
  }
}
```

Alternative approach: separation logic

- separation logic divides resources into “chunks”
- predicates (specifications) are tied to chunks
- works for heaps and other “resources”
- used for concurrency, distributed systems, ...